Q1. Student Performance and Correlation Analysis A college collects data on students' study hours, attendance, and examination marks.

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({ "Study_Hours": [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 2, 4, 6, 8, 3 ],
                "Attendance": [ 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 58, 68, 78, 88, 62 ],
                "Marks": [ 30, 35, 45, 50, 60, 65, 75, 82, 90, 98,  38, 52, 68, 85, 42]
                    })
```

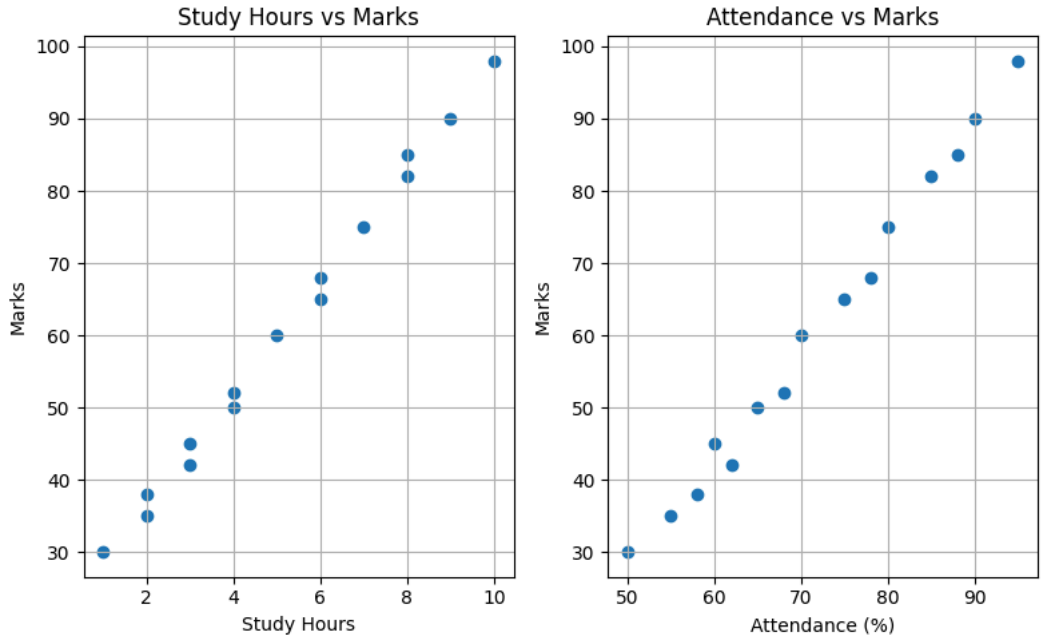
```
# (a) Create scatter plots for Study Hours vs Marks and Attendance vs Marks in separate subplots. Calculate Pearson's
#  correlation coefficient for both pairs and determine which feature has the stronger linear association with marks. Suppo
r_study = df["Study_Hours"].corr(df["Marks"])
r_attendance = df["Attendance"].corr(df["Marks"])
print(r_study,4)
print(r_attendance,4)
fig, axes = plt.subplots(1, 2, figsize=(8, 5))

axes[0].scatter(df["Study_Hours"], df["Marks"])
axes[0].set_title("Study Hours vs Marks")
axes[0].set_xlabel("Study Hours")
axes[0].set_ylabel("Marks")
axes[0].grid(True)

axes[1].scatter(df["Attendance"], df["Marks"])
axes[1].set_title("Attendance vs Marks")
axes[1].set_xlabel("Attendance (%)")
axes[1].set_ylabel("Marks")
axes[1].grid(True)

plt.tight_layout()
plt.show()
```

```
0.9979419719885524 4
0.9957268450507293 4
```




```
# (b) Add a hypothetical student with 1 study hour, 95% attendance, and 98 marks. Recalculate both
#  correlations and visualize the updated data. Explain how this observation affects the correlation coefficients and why a

new_student = {
    "Study_Hours": 1,
    "Attendance": 95,
    "Marks": 98
}

updated_df = pd.concat(
    [df, pd.DataFrame([new_student])],
    ignore_index=True
)

new_r_study = updated_df["Study_Hours"].corr(updated_df["Marks"])
new_r_attendance = updated_df["Attendance"].corr(updated_df["Marks"])
```

```
print("Updated correlation values:")
print("Study Hours vs Marks:", round(new_r_study, 4))
print("Attendance vs Marks:", round(new_r_attendance, 4))

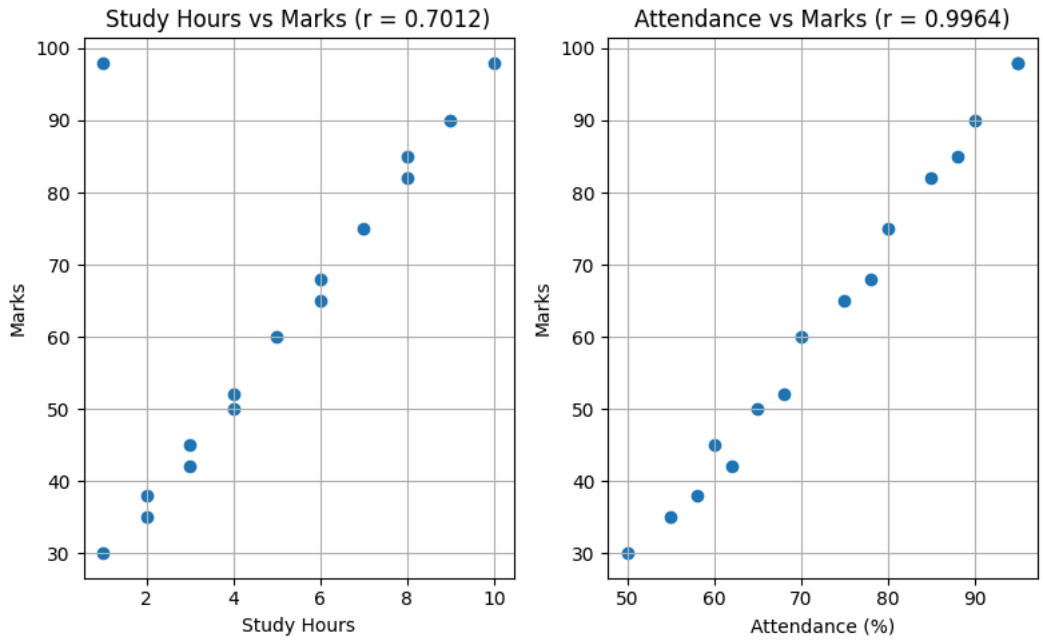
fig, axes = plt.subplots(1, 2, figsize=(8, 5))

axes[0].scatter(
    updated_df["Study_Hours"],
    updated_df["Marks"]
)
axes[0].set_title(
    f"Study Hours vs Marks (r = {new_r_study:.4f})"
)
axes[0].set_xlabel("Study Hours")
axes[0].set_ylabel("Marks")
axes[0].grid(True)

axes[1].scatter(
    updated_df["Attendance"],
    updated_df["Marks"]
)
axes[1].set_title(
    f"Attendance vs Marks (r = {new_r_attendance:.4f})"
)
axes[1].set_xlabel("Attendance (%)")
axes[1].set_ylabel("Marks")
axes[1].grid(True)

plt.tight_layout()
plt.show()
```

```
Updated correlation values:
Study Hours vs Marks: 0.7012
Attendance vs Marks: 0.9964
```



Q2. Product Sales: Bar Plot and Business Decisions An electronics store records its sales and revenue across six products.

```
# (a) Create a figure containing two bar plots comparing Units Sold and Revenue by product. Identify
#  the highest and lowest products for each metric, calculate each product's percentage contribution to total revenue,
# and explain why the product with the highest unit sales may not generate the highest revenue.
df = pd.DataFrame({"Product": ["Laptop", "Mobile", "Tablet", "Headphones", "Monitor", "Keyboard"],
  "Units_Sold": [45, 120, 60, 90, 35, 100],
"Revenue": [2250000, 1800000, 600000, 270000, 525000, 200000]})

total_revenue = df["Revenue"].sum()
df["Revenue_Percentage"] = (
    df["Revenue"] / total_revenue * 100
)

highest_units = df.loc[df["Units_Sold"].idxmax()]
lowest_units = df.loc[df["Units_Sold"].idxmin()]

highest_revenue = df.loc[df["Revenue"].idxmax()]
```

```
lowest_revenue = df.loc[df["Revenue"].idxmin()]

print("Total Revenue: Rs.", total_revenue)
print("\nRevenue contribution:")
print(df[["Product", "Revenue", "Revenue_Percentage"]]
      .to_string(index=False, formatters={
          "Revenue_Percentage": "{:.2f}%".format
      }))

print("\nHighest units sold:", highest_units["Product"])
print("Lowest units sold:", lowest_units["Product"])
print("Highest revenue:", highest_revenue["Product"])
print("Lowest revenue:", lowest_revenue["Product"])

fig, axes = plt.subplots(1, 2, figsize=(8, 5))

axes[0].bar(df["Product"], df["Units_Sold"])
axes[0].set_title("Units Sold by Product")
axes[0].set_xlabel("Product")
axes[0].set_ylabel("Units Sold")
axes[0].tick_params(axis="x", rotation=45)

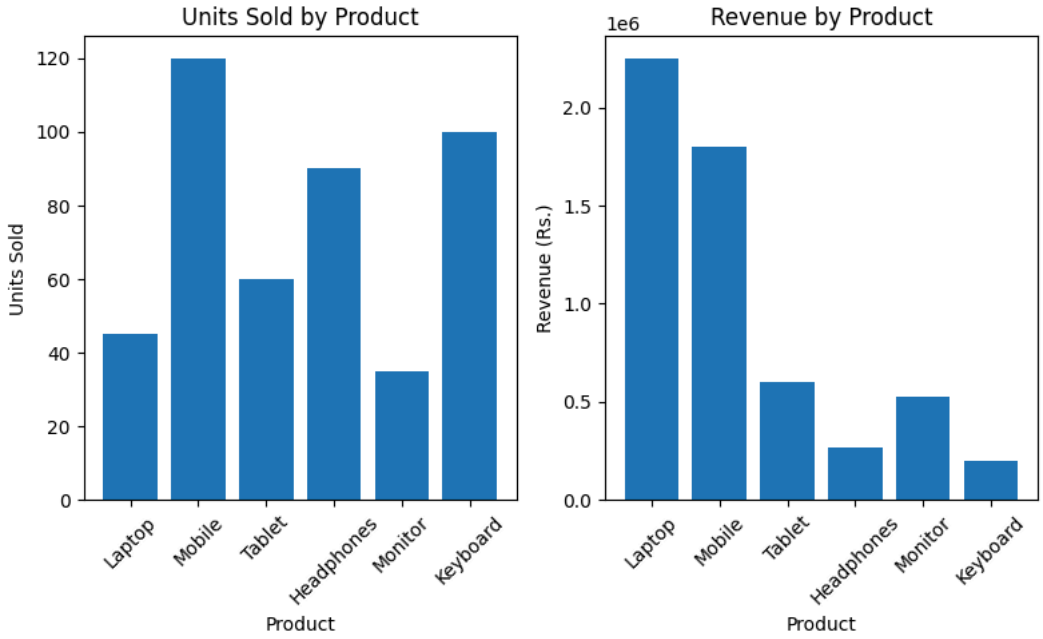
axes[1].bar(df["Product"], df["Revenue"])
axes[1].set_title("Revenue by Product")
axes[1].set_xlabel("Product")
axes[1].set_ylabel("Revenue (Rs.)")
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

```
Total Revenue: Rs. 5645000

Revenue contribution:
   Product  Revenue Revenue_Percentage
    Laptop  2250000             39.86%
    Mobile  1800000             31.89%
    Tablet   600000             10.63%
Headphones   270000              4.78%
   Monitor   525000              9.30%
  Keyboard   200000              3.54%

Highest units sold: Mobile
Lowest units sold: Monitor
Highest revenue: Laptop
Lowest revenue: Keyboard
```




```
# b) Calculate revenue per unit for every product and create a bar plot ranking products by this metric
# . Use Python to identify products with high unit sales but comparatively low revenue per unit, and recommend how the stor
df["Revenue_Per_Unit"] = df["Revenue"] / df["Units_Sold"]

ranked_df = df.sort_values(
    by="Revenue_Per_Unit",
    ascending=False
)
```

```
print("Products ranked by revenue per unit:")
print(
    ranked_df[["Product", "Units_Sold", "Revenue",
               "Revenue_Per_Unit"]].to_string(
        index=False,
        formatters={
            "Revenue_Per_Unit": lambda x: f"Rs. {x:,.2f}"
        }
    )
)

average_units = df["Units_Sold"].mean()
average_revenue_per_unit = df["Revenue_Per_Unit"].mean()

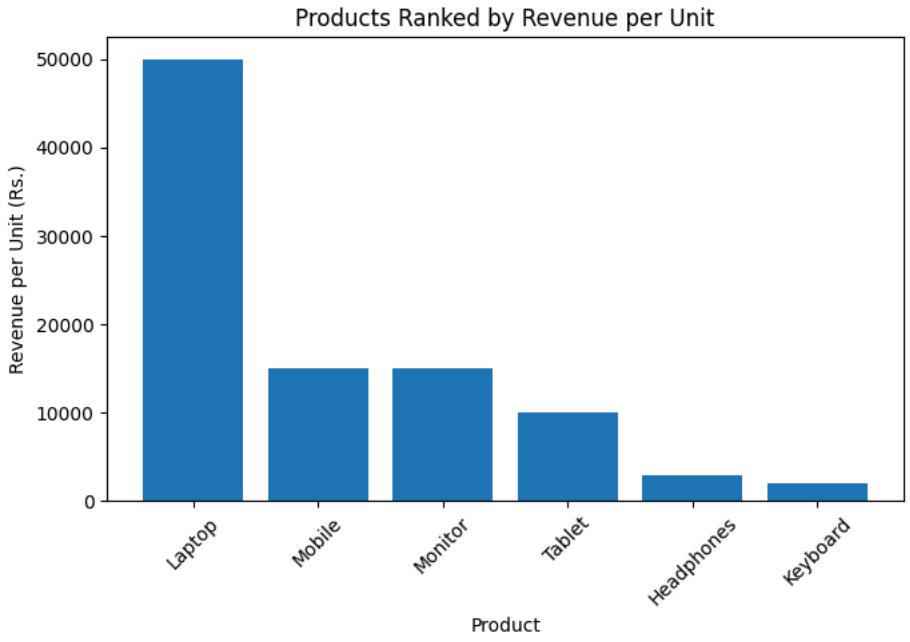
print("\nProducts with above-average units sold")
print("and below-average revenue per unit:")

for _, row in df.iterrows():
    if (row["Units_Sold"] > average_units and
            row["Revenue_Per_Unit"] < average_revenue_per_unit):
        print(row["Product"])

plt.figure(figsize=(7, 5))
plt.bar(ranked_df["Product"], ranked_df["Revenue_Per_Unit"])
plt.title("Products Ranked by Revenue per Unit")
plt.xlabel("Product")
plt.ylabel("Revenue per Unit (Rs.)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Products ranked by revenue per unit:
   Product  Units_Sold  Revenue Revenue_Per_Unit
    Laptop          45  2250000    Rs. 50,000.00
    Mobile         120  1800000    Rs. 15,000.00
   Monitor          35   525000    Rs. 15,000.00
    Tablet          60   600000    Rs. 10,000.00
Headphones          90   270000     Rs. 3,000.00
  Keyboard         100   200000     Rs. 2,000.00

Products with above-average units sold
and below-average revenue per unit:
Mobile
Headphones
Keyboard
```



Q3. Correlation Investigation Using the Tips Dataset Use Seaborn's built-in tips dataset, which contains restaurant bills, tips, dining times, and party sizes.

```
# (a) Create scatter plots for total_bill vs tip and size vs tip. Calculate the Pearson correlation coefficients
#  for both relationships. Identify which pair has the stronger linear association and explain how the scatter plots suppor

import seaborn as sns
df = sns.load_dataset("tips")
```

```
corr_bill_tip = df["total_bill"].corr(df["tip"])
corr_size_tip = df["size"].corr(df["tip"])

print("Correlation (Total Bill vs Tip):", round(corr_bill_tip, 4))
print("Correlation (Party Size vs Tip):", round(corr_size_tip, 4))

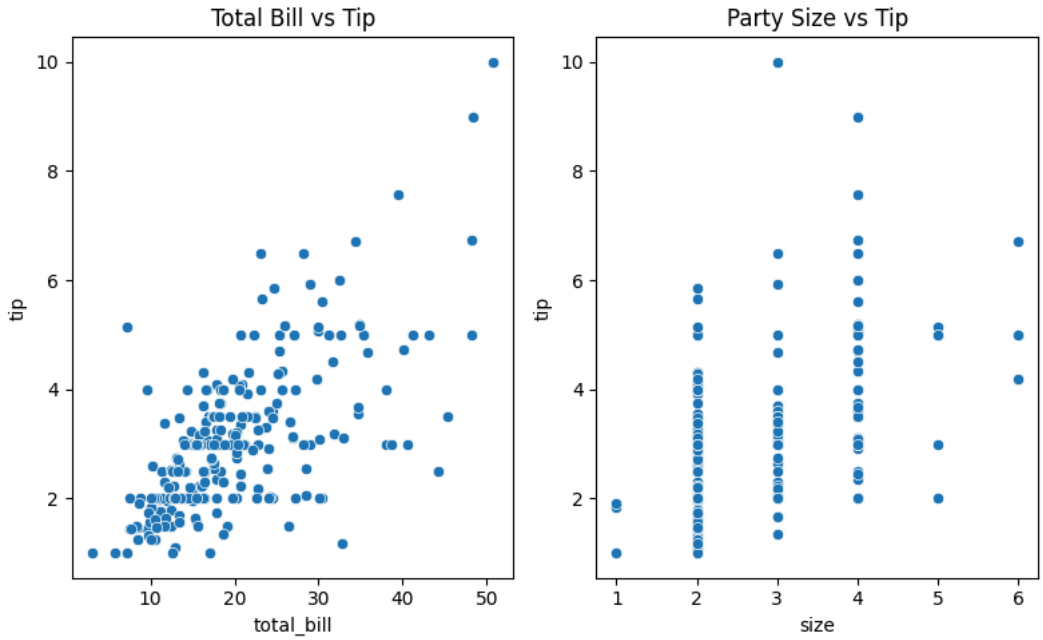
fig, axes = plt.subplots(1, 2, figsize=(8, 5))

sns.scatterplot(data=df, x="total_bill", y="tip", ax=axes[0])
axes[0].set_title("Total Bill vs Tip")

sns.scatterplot(data=df, x="size", y="tip", ax=axes[1])
axes[1].set_title("Party Size vs Tip")

plt.tight_layout()
plt.show()
```

```
Correlation (Total Bill vs Tip): 0.6757
Correlation (Party Size vs Tip): 0.4893
```




```
# (b) Group the dataset by party size and calculate the average tip for each group. Create a bar
#  plot of the average tip by party size, then compare the pattern in the grouped bar plot with the correlation
#  between size and tip. Explain why grouped averages and individual-level correlation can tell different stories.

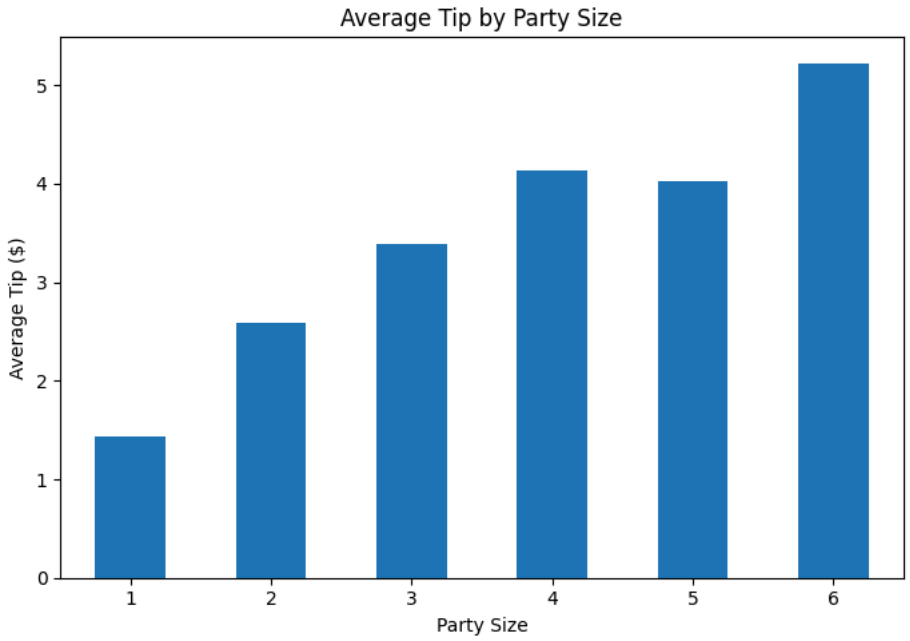
df = sns.load_dataset("tips")

avg_tip = df.groupby("size")["tip"].mean()

print("Average tip by party size:")
print(avg_tip.round(2))
avg_tip.plot(kind="bar", figsize=(7, 5))

plt.title("Average Tip by Party Size")
plt.xlabel("Party Size")
plt.ylabel("Average Tip ($)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
Average tip by party size:
size
1    1.44
2    2.58
3    3.39
4    4.14
5    4.03
6    5.22
Name: tip, dtype: float64
```



Q4. Comparing Positive, Negative, and Weak Correlations A marketing analyst investigates advertising expenditure and sales across different campaigns.

```
# (a) Create scatter plots for Advertising Cost vs Sales and Advertising Cost vs Customer Satisfaction.
#  Calculate both Pearson correlation coefficients, classify each relationship as positive, negative, or weak, and justify y
df = pd.DataFrame({"Campaign": [ "A", "B", "C", "D", "E", "F", "G", "H", "I", "J"    ],
 "Advertising_Cost": [        10, 15, 20, 25, 30, 35, 40, 45, 50, 55    ],
"Sales": [ 90, 80, 75, 70, 65, 60, 55, 50, 45, 40    ],
"Customer_Satisfaction": [ 60, 75, 55, 85, 65, 70, 50, 80, 60, 75    ]})

corr_sales = df["Advertising_Cost"].corr(df["Sales"])
corr_satisfaction = df["Advertising_Cost"].corr(
    df["Customer_Satisfaction"]
)

print("Advertising Cost vs Sales:", round(corr_sales, 4))
print("Advertising Cost vs Customer Satisfaction:",
      round(corr_satisfaction, 4))

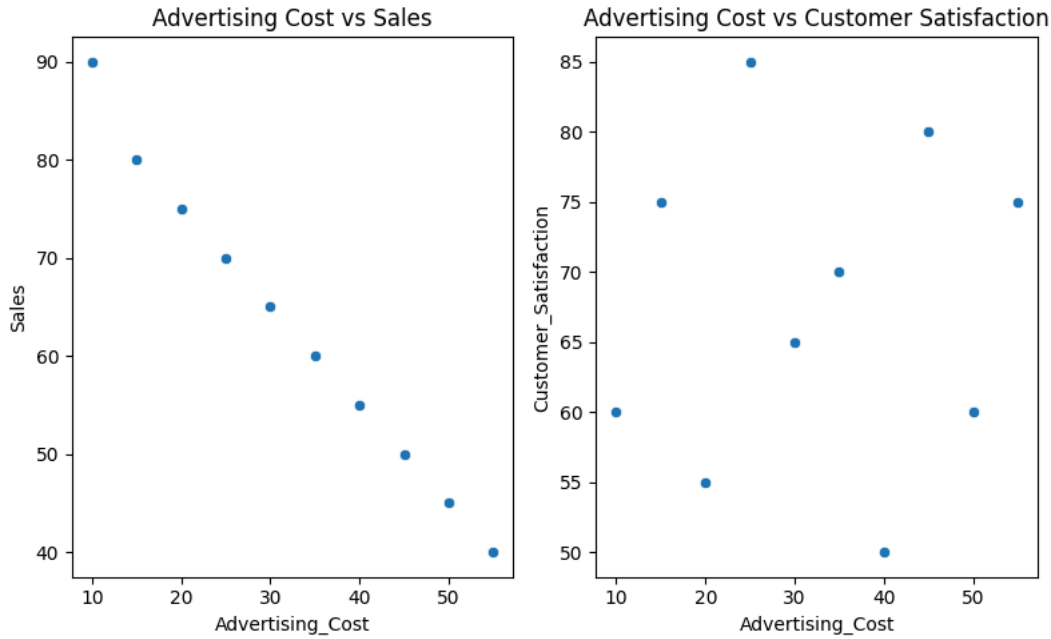
fig, axes = plt.subplots(1, 2, figsize=(8, 5))

sns.scatterplot(data=df, x="Advertising_Cost", y="Sales", ax=axes[0])
axes[0].set_title("Advertising Cost vs Sales")

sns.scatterplot(
    data=df,
    x="Advertising_Cost",
    y="Customer_Satisfaction",
    ax=axes[1]
)
axes[1].set_title("Advertising Cost vs Customer Satisfaction")

plt.tight_layout()
plt.show()
```

```
Advertising Cost vs Sales: -0.9965
Advertising Cost vs Customer Satisfaction: 0.0888
```




```
# (b) Add a new campaign with an advertising cost of 100 and sales of 200. Recalculate the correlation
#  between advertising cost and sales, plot the new point, and explain why correlation does not establish causation
# or guarantee that increased advertising will produce increased sales.

new_campaign = pd.DataFrame({
    "Campaign": ["K"],
    "Advertising_Cost": [100],
    "Sales": [200],
    "Customer_Satisfaction": [None]
})

df_new = pd.concat([df, new_campaign], ignore_index=True)

new_corr = df_new["Advertising_Cost"].corr(df_new["Sales"])

print("New correlation:", round(new_corr, 4))

sns.scatterplot(
    data=df_new,
    x="Advertising_Cost",
    y="Sales"
)

plt.scatter(
    100, 200,
    color="red",
    s=120,
    label="New Campaign K"
)

plt.title("Advertising Cost vs Sales After Adding New Campaign")
plt.xlabel("Advertising Cost")
plt.ylabel("Sales")
plt.legend()
plt.show()
```

```
New correlation: 0.5684
```

